LAPORAN PENGERJAAN UJI KOMPETENSI KEJURUSAN
SMK MAHARDHIKA BATUJAJAR
TAHUN AJARAN 2025/2026

Laporan ini Disusun untuk Memenuhi Persyaratan Kelulusan Tahun Pelajaran 2025/2026

KOMPETENSI KEAHLIAN	: REKAYASA PERANGKAT LUNAK
KELAS	: XII RPL 1
NAMA LENGKAP	: DHEA REVALINA PUTRI
NIS	: 2324.10.355

PROGRAM STUDI KEAHLIAN REKAYASA PERANGKAT LUNAK (RPL)
SMK MAHARDHIKA BATUJAJAR KAB. BANDUNG BARAT
2025/2026

LAPORAN PENGERJAAN UJI KOMPETENSI KEJURUAN
SMK MAHARDHIKA BATUJAJAR
TAHUN PELAJARAN 2025-2026

KOMPETENSI KEAHLIAN	: Rekayasa Perangkat Lunak
KELAS	: XII RPL 1
NAMA LENGKAP	: Dhea Revalina Putri
NIS	: 2324.10.355
PAKET SOAL	: P2
JENIS APLIKASI	: Web
ALOKASI WAKTU	: 24 jam

---

1. PENJADWALAN KEGIATAN

KEGIATAN	ALOKASI WAKTU											
	JANUARI	FEBRUARI				MARET					APRIL	
	28 - 29	3 - 5	10 - 12	17 - 19	24 - 26	3 - 5	10 - 12	17 - 19	24 - 26	31	1 - 2	7 - 9
Analisis Soal												
Persiapan Hardware												
Persiapan Software												
Pembuatan Gantt Chart												
Perancangan sistem												
Perancangan Database												
Perancangan Desain UI												
Implementasi Database												
Implementasi Design UI												
Implementasi Kode Program												
Pengujian Perangkat Lunak												
Pembuatan Dokumen Manual Book												
Finalisasi Dokumen dan Program												
Dokumentasi												
Github												

2. PERSIAPAN PERANGKAT KERAS

System Information

Current Date/Time:	04 April 2026, 12:35:23
Computer Name:	DESKTOP-OBE6C21
Operating System:	Windows 10 Pro 64-bit (10.0, Build 19045)
Language:	English (Regional Setting: English)
System Manufacturer:	HP
System Model:	HP Notebook
BIOS:	F.1D
Processor:	AMD A8-7410 APU with AMD Radeon R5 Graphics (4 CPUs), ~2.2GHz
Memory:	4096MB RAM
Page file:	2974MB used, 7618MB available
DirectX Version:	DirectX 12

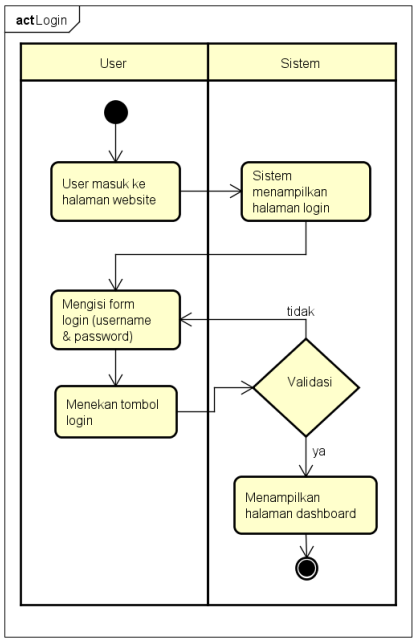
3. PERSIAPAN PERANGKAT LUNAK

NO	SOFTWARE	FUNGSI
1	Visual Studio Code	Visual Studio Code berfungsi untuk mengimplementasikan sebuah kode program dalam proses pengembangan aplikasi parkir, dalam pengembangan proses, tampilan, dan logika.
2	Laragon	Laragon berfungsi sebagai server web yang digunakan untuk pengembangan database.
3	Chrome	Chrome digunakan sebagai perambanan web utama untuk menampilkan tampilan hasil kode program.
4	Astah Professional	Astah digunakan untuk merancang algoritma yang menjadi alur sistem dari aplikasi parkir.
5	Balsamiq	Balsamiq digunakan untuk merancang tampilan sebelum di implementasikan oleh kode program.
6	Microsoft office	Microsoft office berfungsi untuk Menyusun laporan dan dokumentasi dari aplikasi yang dikembangkan.

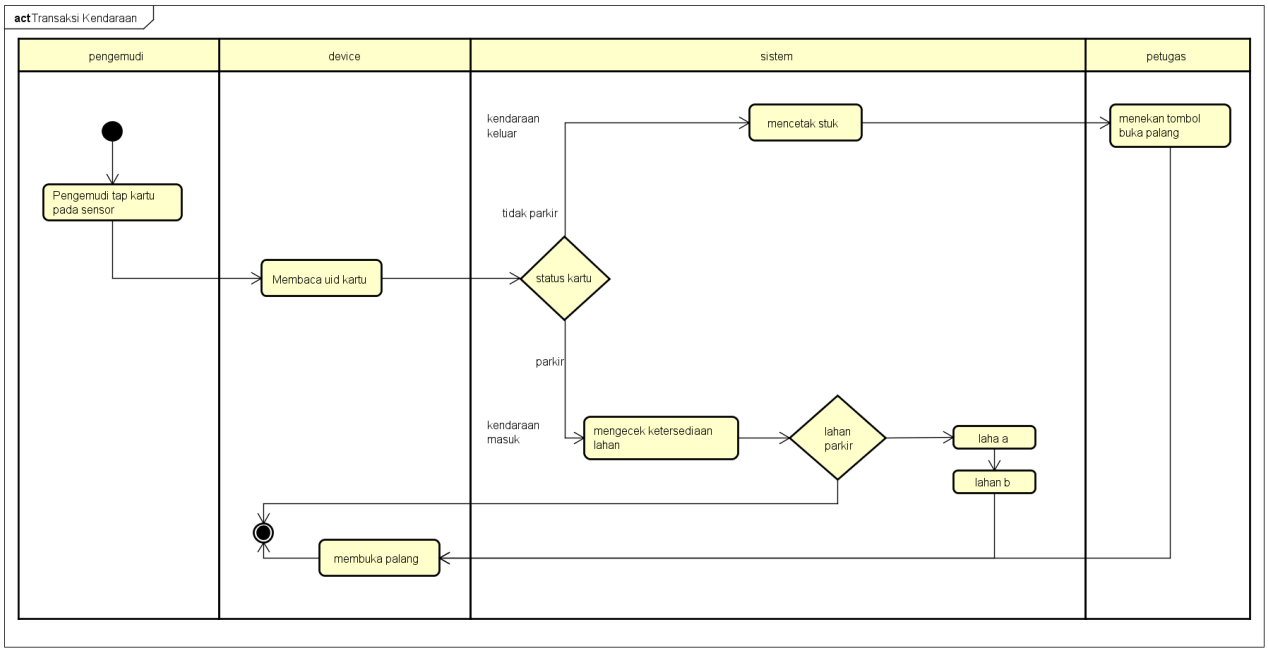
4. PERANCANGAN ALGORITMA

A. Activity Diagram

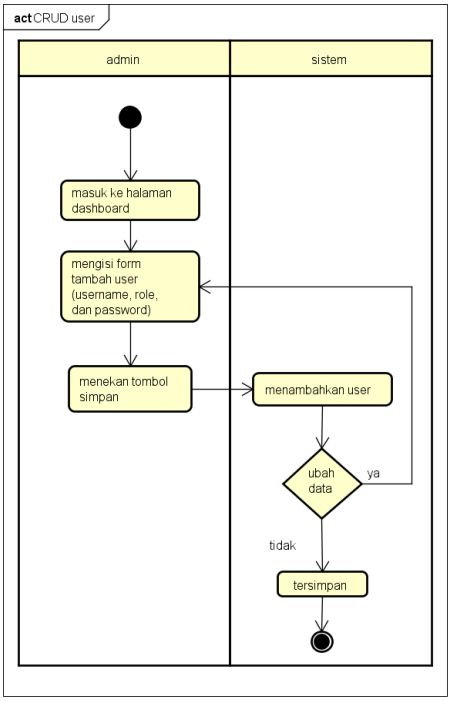
a. Login



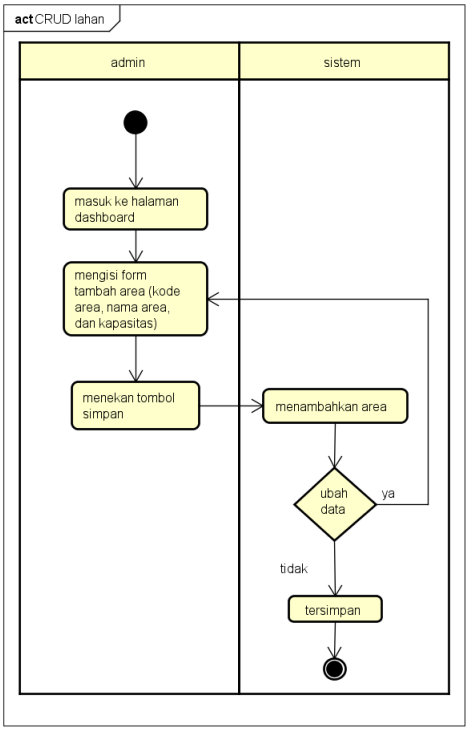
b. Transaksi Kendaraan



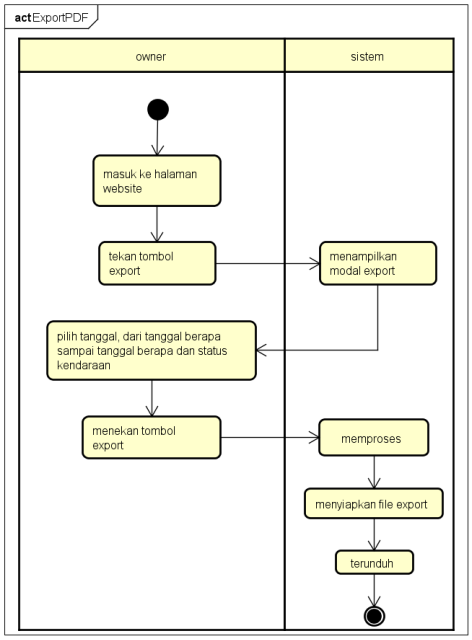
c. CRUD User



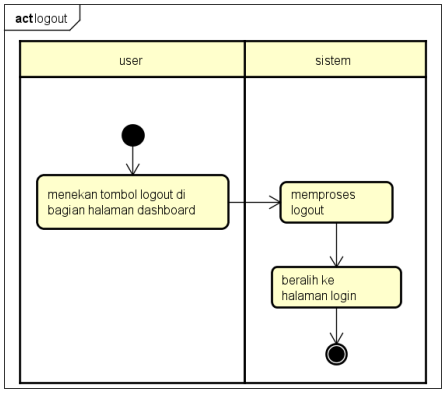
d. CRUD Kapasitas



e. Export PDF

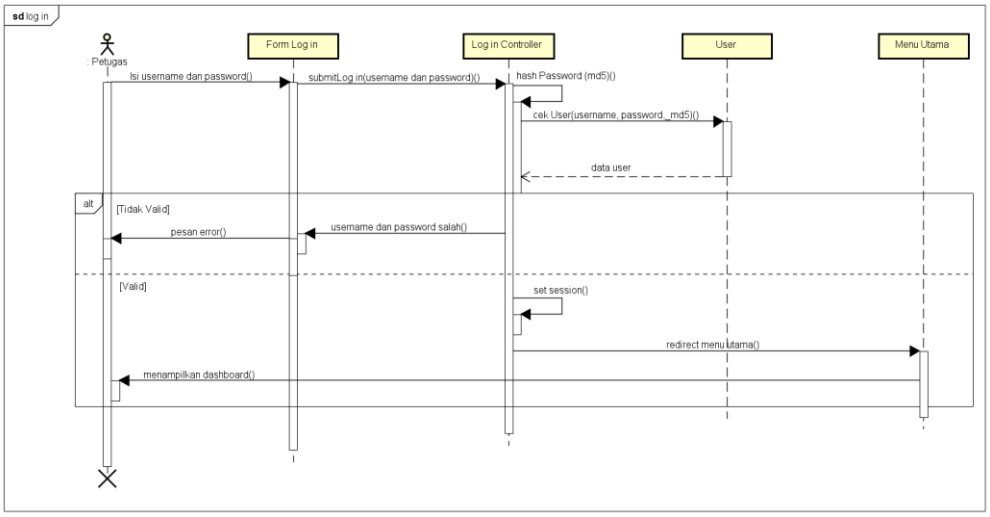


f. Logout

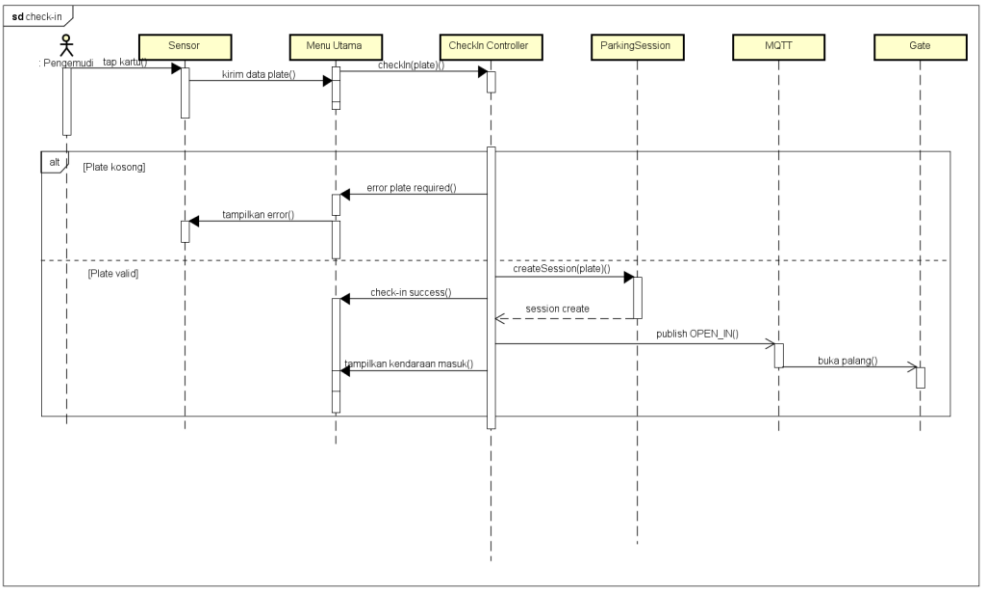


B. Sequence Diagram

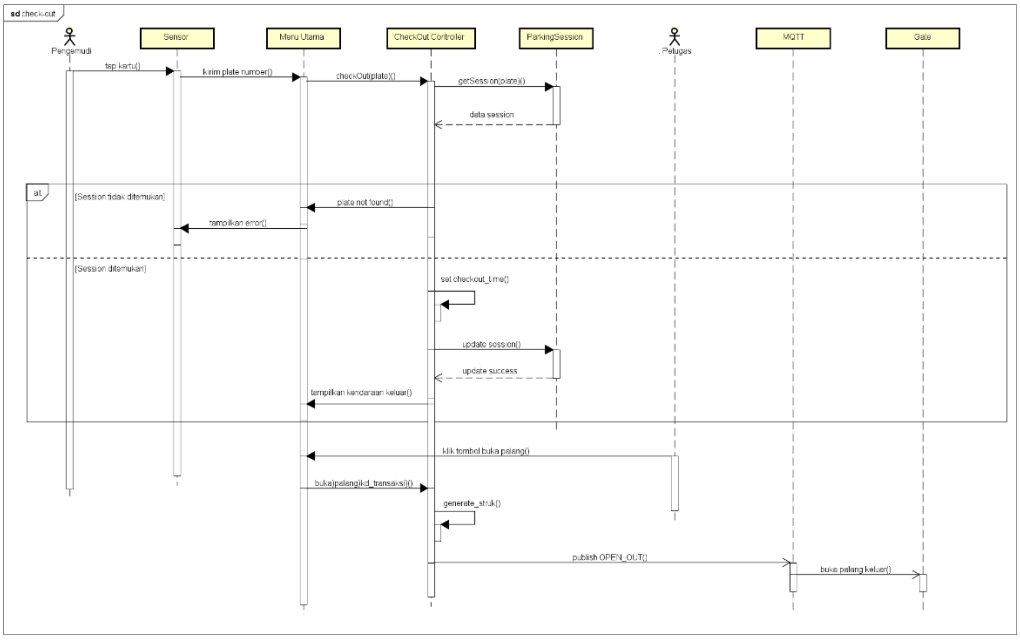
a. Login



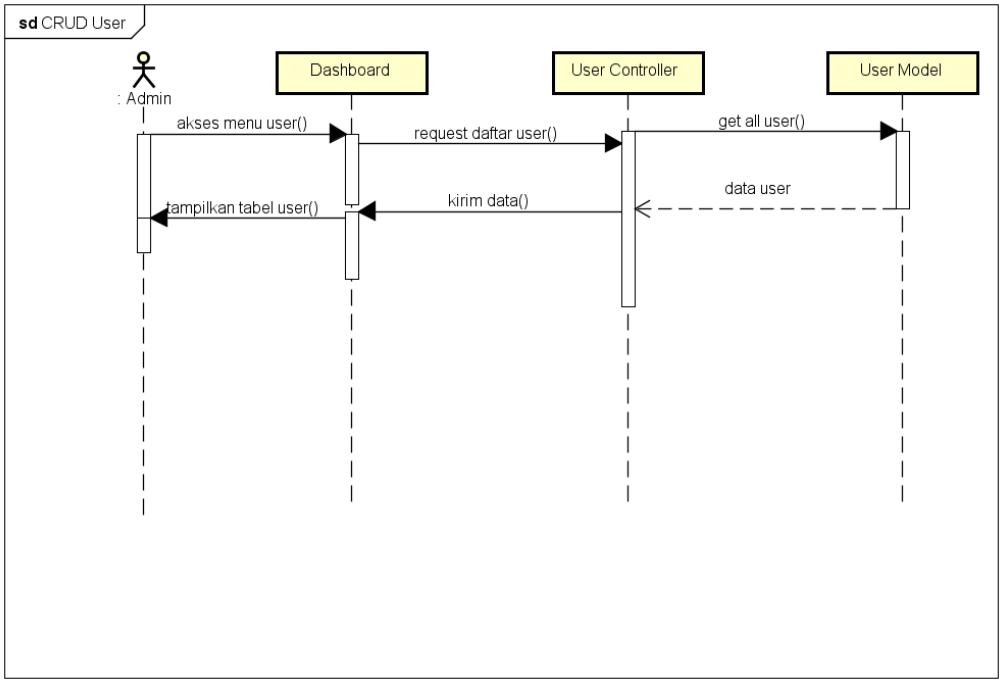
b. Check-in



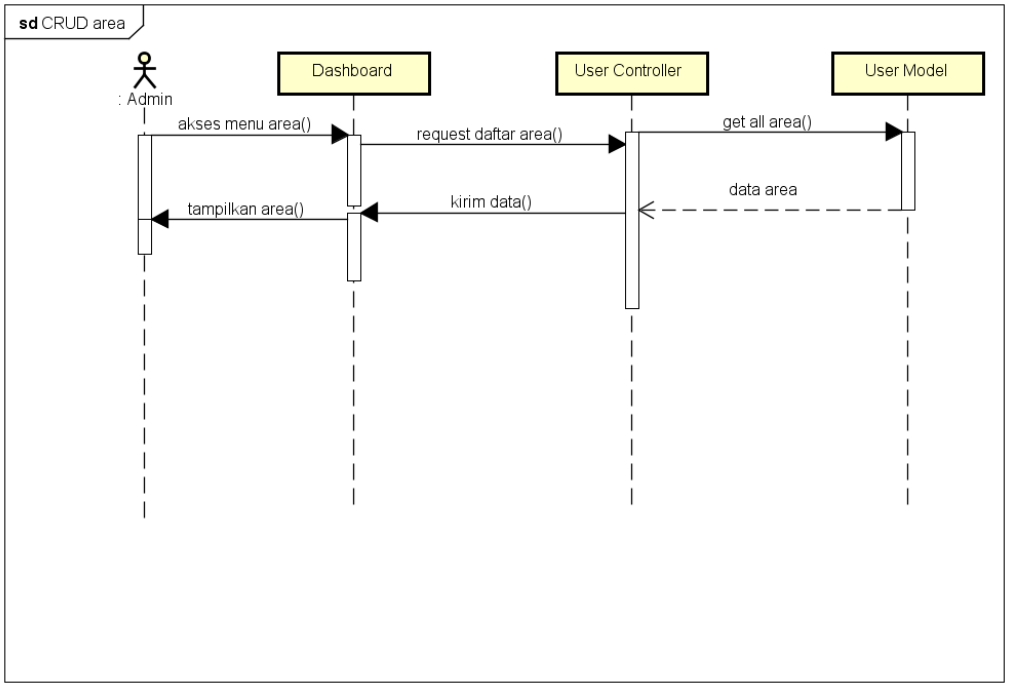
c. Check-out



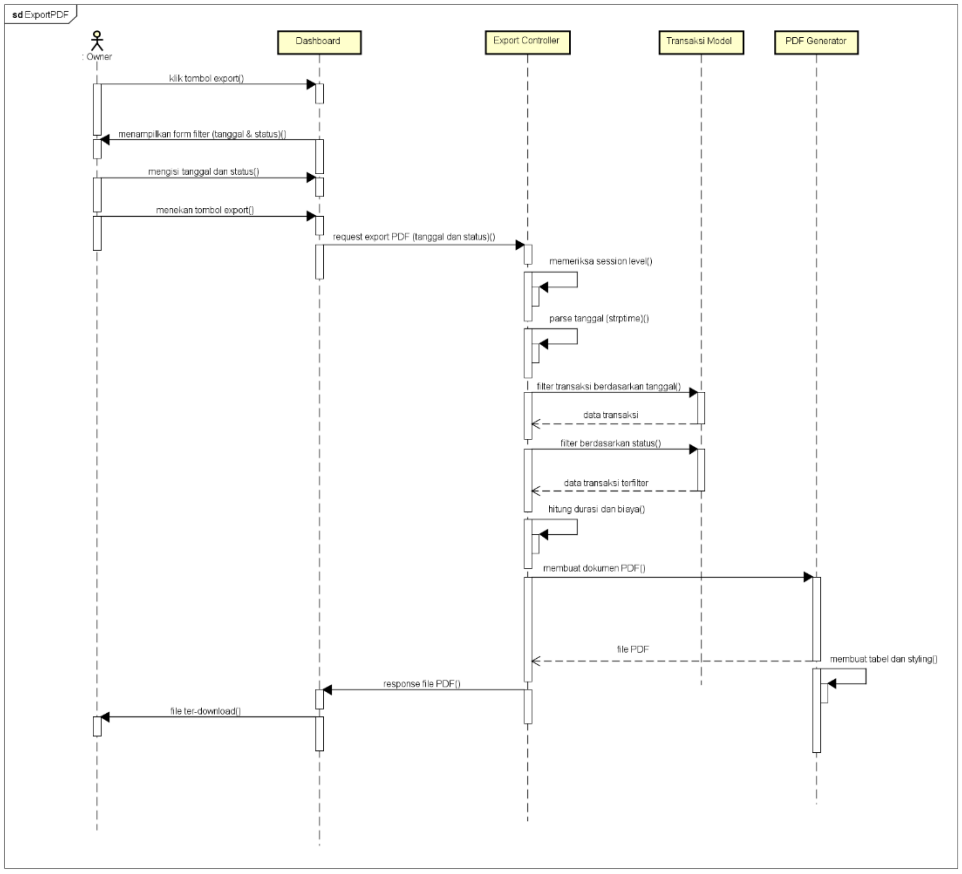
d. CRUD User



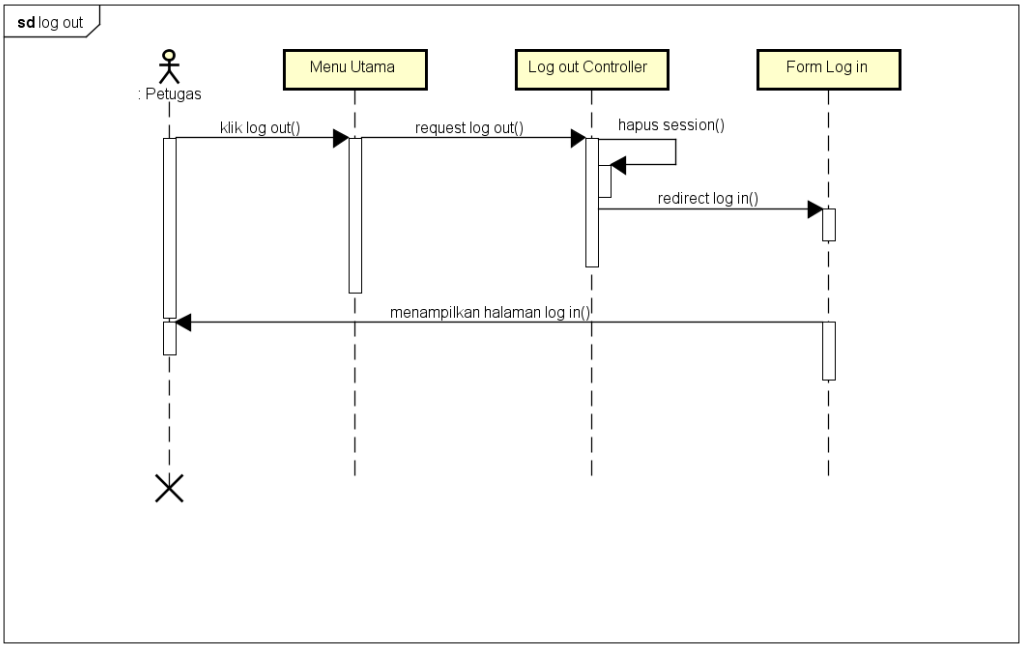
e. CRUD kapasitas



f. Export PDF

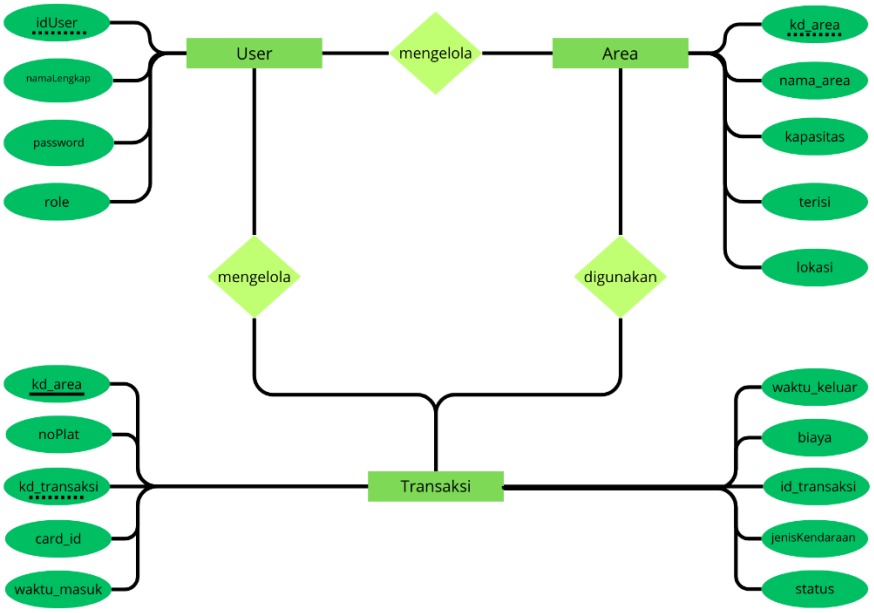


g. Logout

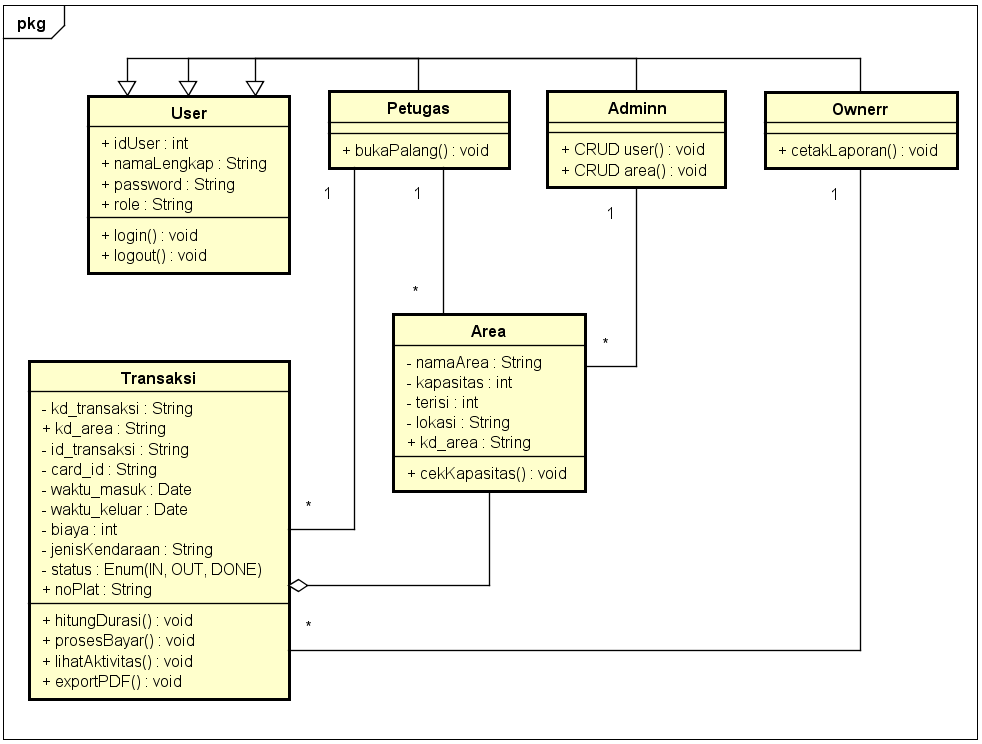


5. PERANCANGAN BASIS DATA

A. ERD



B. Class Diagram



6. PENGISIAN DUMMY DATA

A. Data Pengguna

Id	Nama Lengkap	Username	password	Level
1	erik	erik1	f12537e9605b2b1bf3122bb12a0e24f7	petugas
2	fred	fred2	77064f5bd13e417f564e7d880dc7a536	petugas

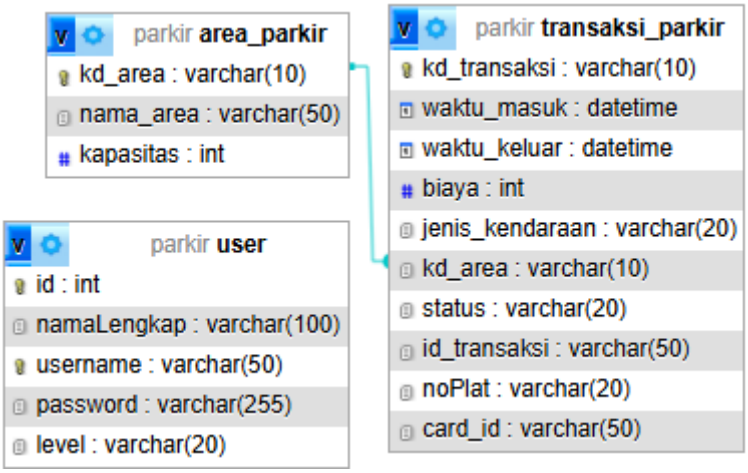
B. Data Pengemudi

Kd_ transaksi	Waktu_ masuk	Waktu_ keluar	biaya	Jenis_ kendaraan	Kd_ area	status	Id_ transaksi	noPlat	Card_ id
B38	06:16:39	NULL	0	Motor	BSP	IN	V5HnS3lI	T 9235 TS	36100207
B42	06:10:27	06:11:12	2000	Motor	BSP	IN	oryXPMtd	I 9601 JG	53063956

C. Data Area

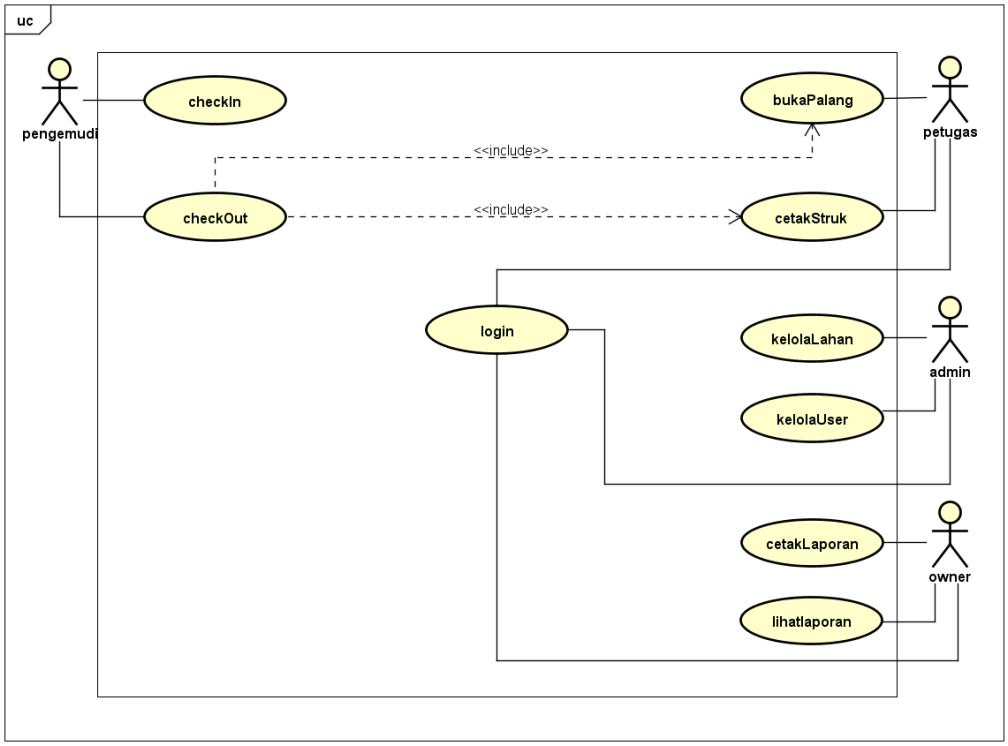
kd_area	nama_area	kapasitas
BSP	DOR	50
LKW	KOL	50
TXS	WEW	50

7. PEMBUATAN BASIS DATA



8. PERANCANGAN TAMPILAN ANTARMUKA

A. Use Case Diagram



B. Skenario Use Case

a. Login

Identifikasi	
Nama	Masuk
IDE	ZH-001
Deskripsi	Masuk ke halaman login dan halaman beranda
Author	User
Main Course	
Kondisi Awal	**Reaksi Sistem**
1. Masuk ke halaman website	
	2. Menampilkan form login
3. Mengisi form login (username & password)	
4. Mengklik tombol login	
	5. Menampilkan halaman dashboard
Alternative Course	
1. Masuk ke halaman website	
	2. Menampilkan form login
3. Mengisi form login (username & password	
4. Mengklik tombol enter pada keyboard	
	5. Menampilkan halaman dashboard
Exception Course	
1. Masuk ke halaman website	
	2. Menampilkan form login
3. Salah mengisi form login (username & password	
4. Mengklik tombol login	
	5. Menampilkan pesan “username dan password salah!”

b. CRUD User (Tambah)

Identifikasi	
Nama	CRUD User
IDE	ZH-002
Deskripsi	Menambah, mengubah, dan menghapus data user
Author	Admin
Main Course	
Kondisi Awal	**Reaksi Sistem**
1. Masuk ke halaman website	
	2. Menampilkan halaman dashboard
3. Mengisi form user (username, role & password)	
4. Mengklik tombol simpan	
	5. Menyimpan data user baru

c. CRUD Area

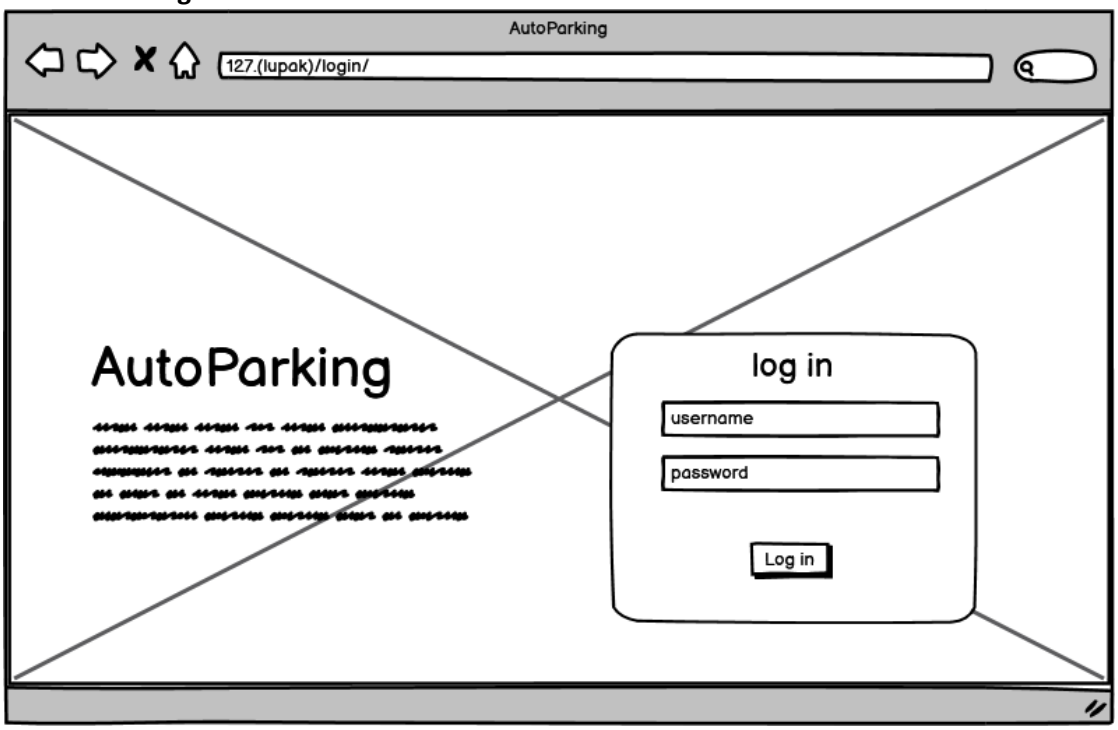
Identifikasi	
Nama	CRUD Area
IDE	ZH-003
Deskripsi	Menambah, mengubah, dan menghapus data area
Author	Admin

Main Course	
Kondisi Awal	Reaksi Sistem
1. Masuk ke halaman website	
	2. Menampilkan halaman dashboard
3. Mengisi form user (nama area & kapasitas)	
4. Mengklik tombol simpan	
	5. Menyimpan data area baru

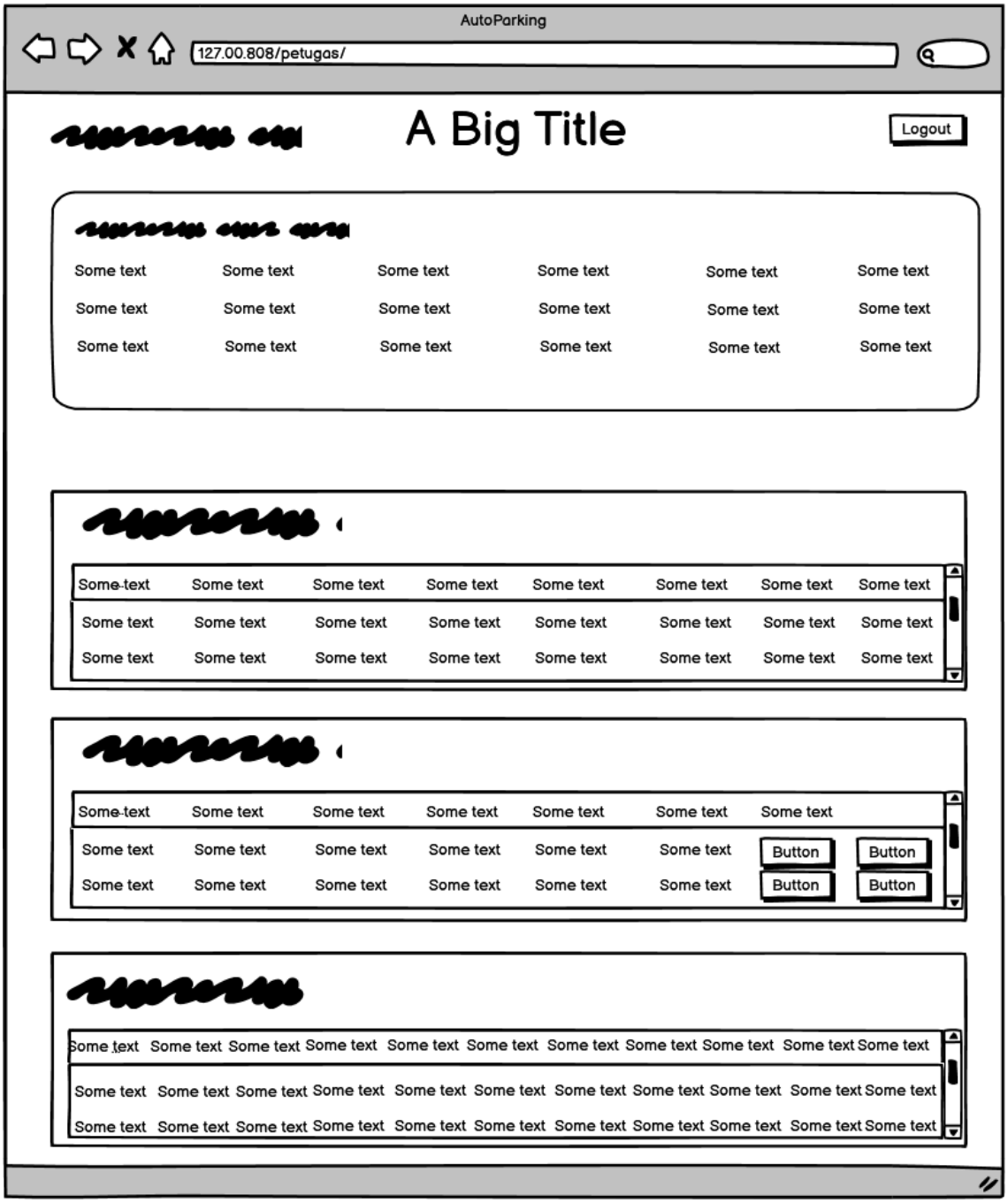
C. Tampilan Aplikasi

1) Mockup Aplikasi

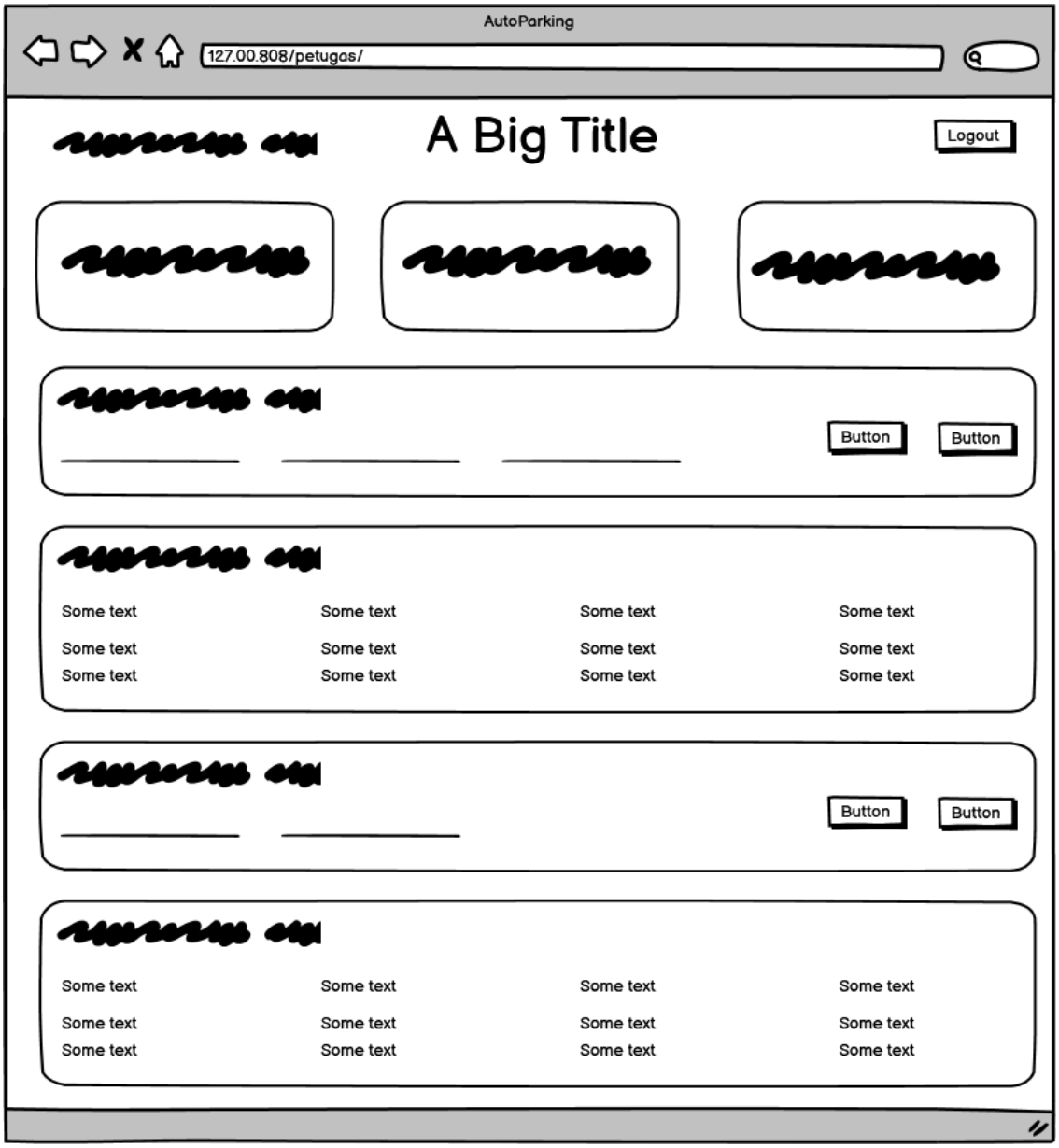
a. Login



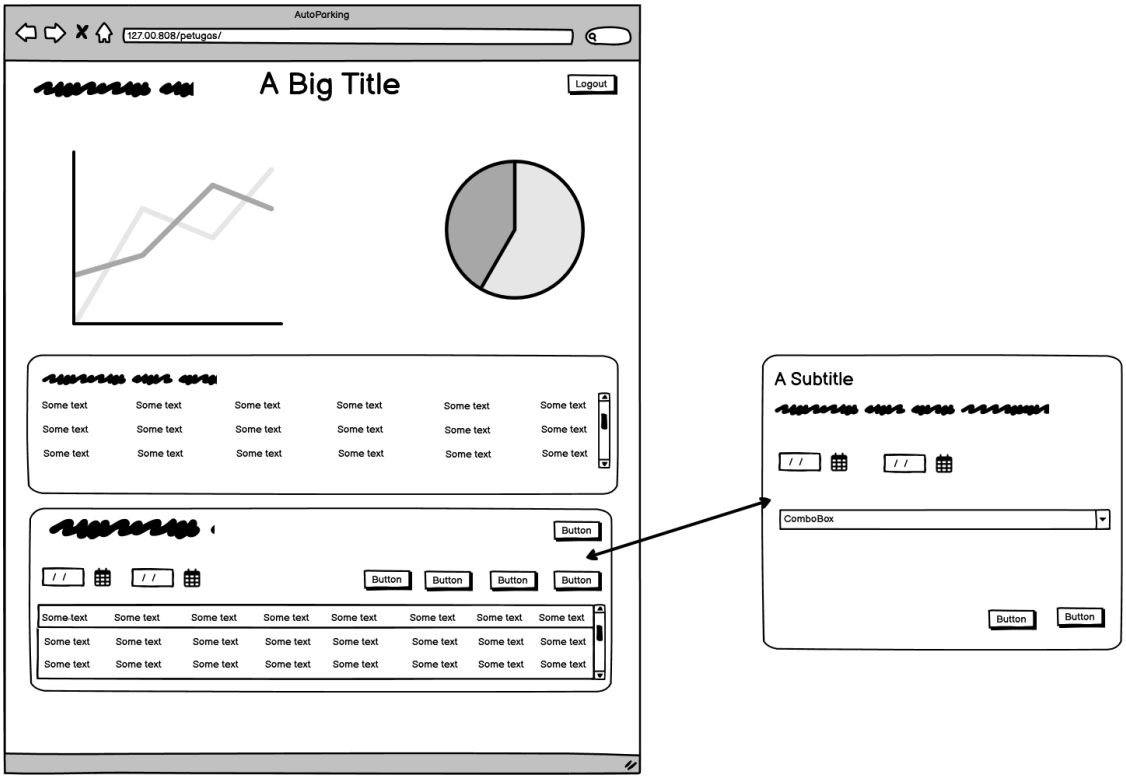
b. Petugas


AutoParking
127.00.808/petugas/
A Big Title
Logout
Some text
Button

c. **Admin**

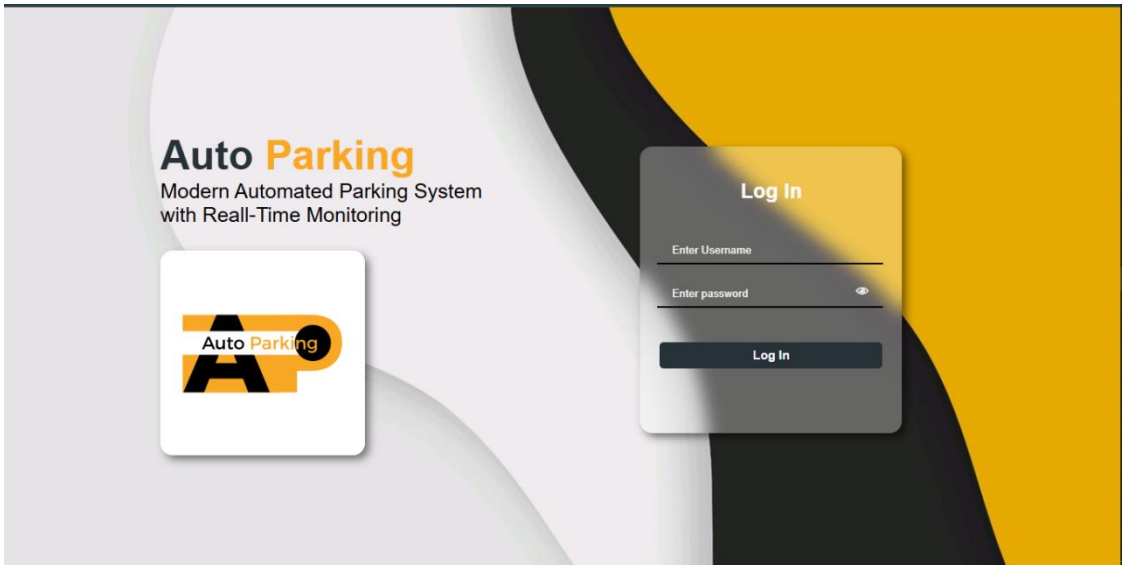

AutoParking
127.00.808/petugas/
A Big Title
Logout
Button
Button
Some text
Button
Button
Some text

d. Owner



2) Implementasi Aplikasi

a. Login



b. Dashboard Petugas

Selamat Datang, fred2 **Auto Parking** Logout

Area Parkir

Kode	Nama Area	Kapasitas	Sedang Parkir	Selesai	Tersedia
BSP	KOL	50	2	16	48
LKW	DOR	50	0	0	50
SPD	VFT	30	0	0	30
TXS	WEW	50	0	0	50

Kendaraan Sedang Parkir

Area	Kd Transaksi	Card ID	Plat No	Masuk	Durasi	Biaya	Status
BSP	B38	36100207	**T 9235 TS**	06:16:39	89 jam	Rp 178000	IN
BSP	Z17	FEA66A06	**A 6514 JW**	06:13:32	89 jam	Rp 178000	IN

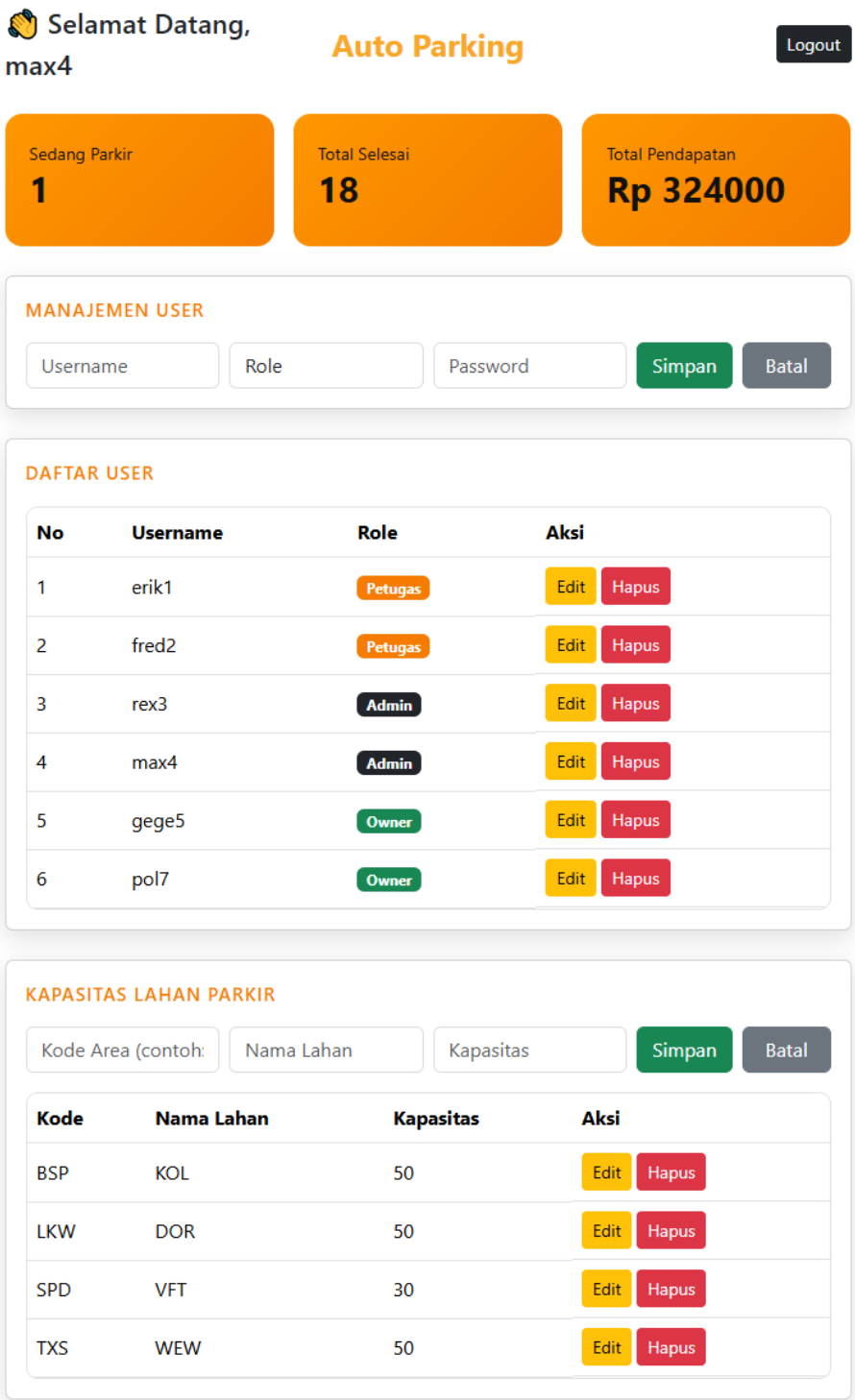
Menunggu Buka Palang Keluar

Area	Kd Transaksi	Card ID	Plat No	Masuk	Keluar	Durasi	Biaya	Status	Aksi
Tidak ada kendaraan yang menunggu keluar									

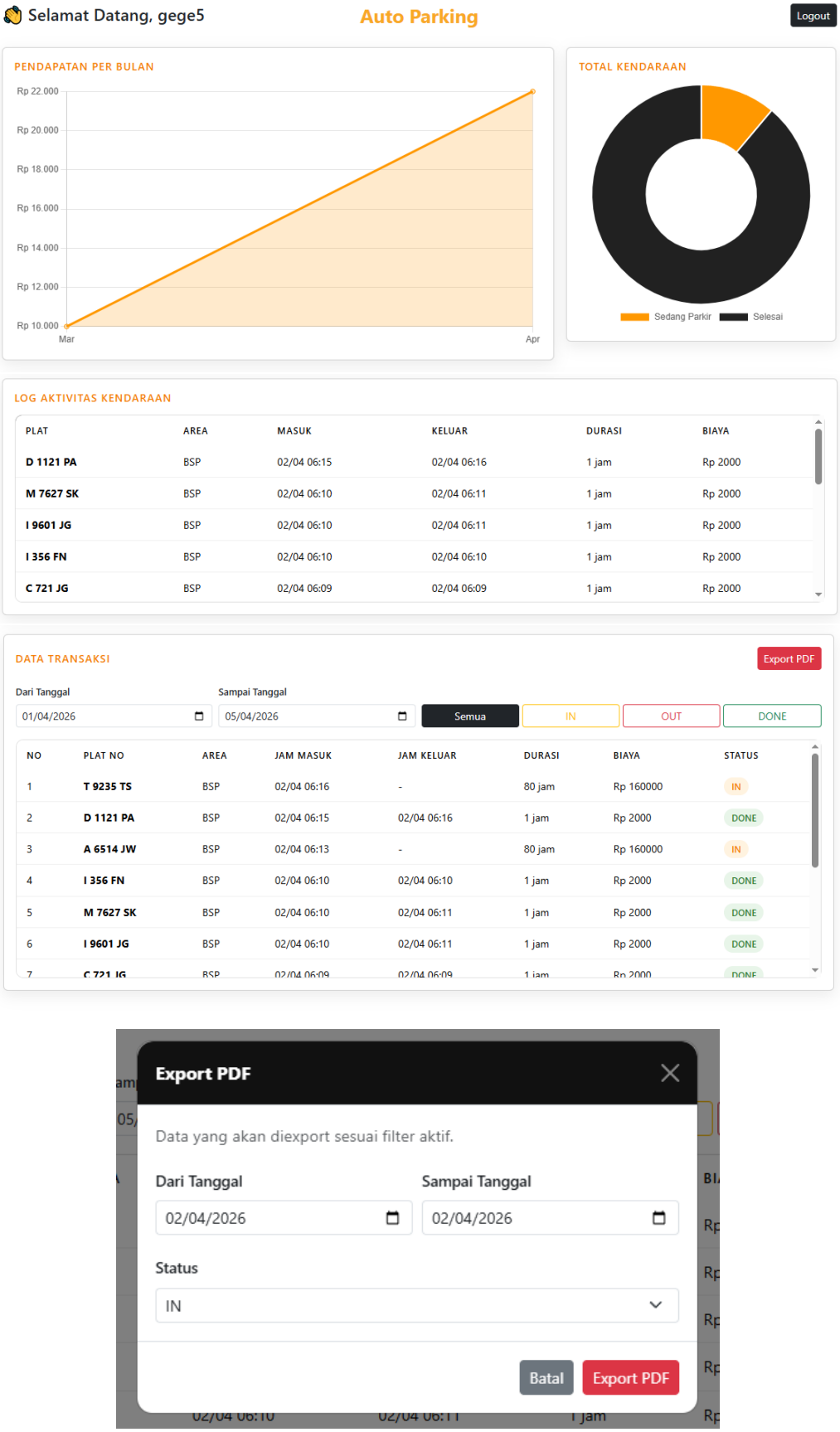
Log Riwayat Transaksi

No	Area	Kd Transaksi	Card ID	Plat No	Masuk	Keluar	Durasi	Biaya	Tanggal	Status
1	**BSP**	X59	36100207	**D 1121 PA**	06:15:49	06:16:04	1 jam	Rp 2000	02/04/2026	DONE
2	**BSP**	O88	FEA66A06	**M 7627 SK**	06:10:36	06:11:20	1 jam	Rp 2000	02/04/2026	DONE
3	**BSP**	B42	53063956	**I 9601 JG**	06:10:27	06:11:12	1 jam	Rp 2000	02/04/2026	DONE
4	**BSP**	V87	36100207	**I 356 FN**	06:10:45	06:10:55	1 jam	Rp 2000	02/04/2026	DONE
5	**BSP**	K52	53063956	**C 721 JG**	06:09:02	06:09:16	1 jam	Rp 2000	02/04/2026	DONE
6	**BSP**	Q47	53063956	**M 206 VF**	06:03:42	06:03:58	1 jam	Rp 2000	02/04/2026	DONE
7	**BSP**	L89	FEA66A06	**D 8483 XP**	05:36:12	06:01:52	1 jam	Rp 2000	02/04/2026	DONE
8	**BSP**	N67	53063956	**F 6084 TR**	05:41:18	05:41:27	1 jam	Rp 2000	02/04/2026	DONE
9	**BSP**	D93	53063956	**C 1742 PM**	05:39:32	05:39:48	1 jam	Rp 2000	02/04/2026	DONE

c. **Dashboard Admin**



d. Dashboard Owner



9. PEMBUATAN KODE WEB

A. Views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.http import JsonResponse, HttpResponse
from django.utils import timezone
from django.db import connection
from django.views.decorators.http import require_POST
from .models import TransaksiParkir, AreaParkir, ParkingSession
from .mqtt_client import (
    publish_message,
    approve_keluar, reject_keluar,
    broadcast_petugas_login, broadcast_petugas_logout,
)
import math
import hashlib
import random
import string
from .mqtt_client import pop_struk_terbaru
from .models import TransaksiParkir, AreaParkir, ParkingSession, User
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet
from datetime import datetime
from django.db.models import Sum, Count
from django.db.models.functions import TruncMonth

# ================= HELPER AUTOMATIC =================

def hitung_durasi_biaya(waktu_masuk, waktu_keluar=None):
    waktu_akhir = waktu_keluar if waktu_keluar else timezone.now()
    durasi_jam  = math.ceil((waktu_akhir - waktu_masuk).total_seconds() / 3600) or 1
    return durasi_jam, durasi_jam * 2000

def get_shift(waktu):
    jam = waktu.hour
    if 5 <= jam <= 11:    return "Pagi"
    elif 12 <= jam <= 15: return "Siang"
    elif 16 <= jam <= 17: return "Sore"
    else:                 return "Malam"

def cek_struk(request):
    data = pop_struk_terbaru()
    print(f"[CEK STRUK] dipanggil, ada: {data is not None}")
    if data:
        return JsonResponse({"ada": True, "kd_transaksi": data["kd_transaksi"], "struk": data["struk"]})
    return JsonResponse({"ada": False})

def generate_struk(transaksi, operator):
    durasi_jam, biaya = hitung_durasi_biaya(transaksi.waktu_masuk, transaksi.waktu_keluar)
    shift = get_shift(transaksi.waktu_keluar)
    return f"""STRUK AUTOPARKING
---------------------------------------------
Jenis Transaksi : Cash
ID Card         : {transaksi.card_id}
Masuk           : {transaksi.waktu_masuk.strftime('%Y-%m-%d %H:%M:%S')}
Keluar          : {transaksi.waktu_keluar.strftime('%Y-%m-%d %H:%M:%S')}
Plat Nomor      : {transaksi.noPlat}
Lama Parkir     : {durasi_jam} jam
Biaya Parkir    : Rp {biaya:,}
---------------------------------------------
Operator        : {operator} ({shift})
=============================================
"""

# ================= LOGIN =================
def login_view(request):
    if request.method == 'POST':
        username     = request.POST.get('username')
```

```
        password     = request.POST.get('password')
        password_md5 = hashlib.md5(password.encode()).hexdigest()

        with connection.cursor() as cursor:
            cursor.execute(
                "SELECT username, level FROM user WHERE username=%s AND password=%s",
                [username, password_md5]
            )
            user = cursor.fetchone()

        if user:
            request.session['username'] = user[0]
            request.session['level']    = user[1]
            if user[1] == 'petugas':
                broadcast_petugas_login(user[0])
                return redirect('petugas')
            elif user[1] == 'admin':
                return redirect('admin')
            elif user[1] == 'owner':
                return redirect('owner')


        return render(request, 'login.html', {'error': 'Username atau password salah'})

    return render(request, 'login.html')

# ================= DASHBOARD PETUGAS =================
def petugas_dashboard(request):
    if request.session.get('level') != 'petugas':
        return redirect('login')

    areas     = AreaParkir.objects.all()
    data_area = []
    for area in areas:
        aktif    = TransaksiParkir.objects.filter(kd_area=area, status='IN').count()
        selesai  = TransaksiParkir.objects.filter(kd_area=area, status='DONE').count()
        tersedia = area.kapasitas - aktif
        data_area.append({'area': area, 'aktif': aktif, 'selesai': selesai, 'tersedia': tersedia})

    # Kendaraan sedang parkir (IN)
    kendaraan_masuk = TransaksiParkir.objects.filter(status='IN').order_by('-waktu_masuk')
    for k in kendaraan_masuk:
        k.durasi_display, k.biaya_display = hitung_durasi_biaya(k.waktu_masuk)

    # Kendaraan menunggu buka palang (OUT)
    kendaraan_keluar = TransaksiParkir.objects.filter(status='OUT').order_by('-waktu_keluar')
    for k in kendaraan_keluar:
        k.durasi_display, k.biaya_display = hitung_durasi_biaya(k.waktu_masuk, k.waktu_keluar)

    # Riwayat selesai (DONE)
    log_riwayat = TransaksiParkir.objects.filter(status='DONE').order_by('-waktu_keluar')[:100]
    for index, log in enumerate(log_riwayat, 1):
        log.no_urut = index
        log.durasi_display, _ = hitung_durasi_biaya(log.waktu_masuk, log.waktu_keluar)

    context = {
        'data_area':       data_area,
        'kendaraan_masuk':  kendaraan_masuk,
        'kendaraan_keluar': kendaraan_keluar,
        'log_riwayat':      log_riwayat,
    }
    return render(request, 'petugas/petugas_dashboard.html', context)

# ================= TRANSAKSI CHECK-IN/OUT PETUGAS =================
def check_in(request):
    plate = request.GET.get("plate")
    if not plate:
        return JsonResponse({"error": "Plate number required"})
    ParkingSession.objects.create(plate_number=plate)
```

```
    publish_message("OPEN_IN")
    return JsonResponse({"message": "Check-in success", "plate": plate})

def check_out(request):
    plate = request.GET.get("plate")
    try:
        session = ParkingSession.objects.get(plate_number=plate, is_active=True)
    except ParkingSession.DoesNotExist:
        return JsonResponse({"error": "Plate not found"})
    session.check_out_time = timezone.now()
    session.is_active      = False
    session.save()
    publish_message("OPEN_OUT")
    return JsonResponse({"message": "Check-out success", "plate": plate})

# ================= BUKA PALANG PETUGAS =================
@require_POST
def aksi_buka_keluar(request, kd_transaksi):
    if request.session.get('level') != 'petugas':
        return JsonResponse({"success": False, "message": "Unauthorized"}, status=403)

    success, msg = approve_keluar(kd_transaksi)
    if success:
        try:
            transaksi = TransaksiParkir.objects.get(kd_transaksi=kd_transaksi)
            struk     = generate_struk(transaksi, request.session.get('username', 'Petugas'))
            with open(f"struk_{kd_transaksi}.txt", "w") as f:
                f.write(struk)
        except Exception as e:
            print(f"[STRUK] Error: {e}")

    return JsonResponse({"success": success, "message": msg})

@require_POST
def aksi_tolak_keluar(request, kd_transaksi):
    if request.session.get('level') != 'petugas':
        return JsonResponse({"success": False, "message": "Unauthorized"}, status=403)
    success, msg = reject_keluar(kd_transaksi)
    return JsonResponse({"success": success, "message": msg})

# ================= DASHBOARD ADMIN =================
def admin_dashboard(request):
    if request.session.get('level') != 'admin':
        return redirect('login')

    sedang_parkir = TransaksiParkir.objects.filter(status='IN').count()
    total_selesai = TransaksiParkir.objects.filter(status='DONE').count()

    bulan_ini = timezone.now().replace(day=1, hour=0, minute=0, second=0, microsecond=0)
    transaksi_bulan_ini = TransaksiParkir.objects.filter(status='DONE', waktu_keluar__gte=bulan_ini)
    total_pendapatan = sum(
        hitung_durasi_biaya(t.waktu_masuk, t.waktu_keluar)[1]
        for t in transaksi_bulan_ini
    )

    daftar_user = User.objects.all().order_by('id')
    daftar_area = AreaParkir.objects.all().order_by('kd_area')

    context = {
        'sedang_parkir':    sedang_parkir,
        'total_selesai':    total_selesai,
        'total_pendapatan': total_pendapatan,
        'daftar_user':      daftar_user,
        'daftar_area':      daftar_area,
    }
    return render(request, 'admin/admin_dashboard.html', context)

# ================= CRUD USER ADMIN =================
def tambah_user(request):
```

```
    if request.session.get('level') != 'admin':
        return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

    if request.method == 'POST':
        username     = request.POST.get('username', '').strip()
        role         = request.POST.get('role', '').strip()
        password     = request.POST.get('password', '').strip()

        if not username or not role or not password:
            return JsonResponse({'success': False, 'message': 'Semua kolom harus diisi!'})

        if User.objects.filter(username=username).exists():
            return JsonResponse({'success': False, 'message': 'Username sudah ada!'})

        password_md5 = hashlib.md5(password.encode()).hexdigest()
        User.objects.create(username=username, password=password_md5, level=role, namaLengkap=username)

        return JsonResponse({'success': True, 'message': 'User berhasil ditambahkan!'})

def edit_user(request, user_id):
    if request.session.get('level') != 'admin':
        return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        role     = request.POST.get('role', '').strip()
        password = request.POST.get('password', '').strip()

        if not username or not role:
            return JsonResponse({'success': False, 'message': 'Semua kolom harus diisi!'})

        try:
            user = User.objects.get(id=user_id)
            user.username = username
            user.level    = role
            if password:
                user.password = hashlib.md5(password.encode()).hexdigest()
            user.save()
            return JsonResponse({'success': True, 'message': 'User berhasil diupdate!'})
        except User.DoesNotExist:
            return JsonResponse({'success': False, 'message': 'User tidak ditemukan!'})


def hapus_user(request, user_id):
    if request.session.get('level') != 'admin':
        return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

    if request.method == 'POST':
        try:
            user = User.objects.get(id=user_id)
            user.delete()
            return JsonResponse({'success': True, 'message': 'User berhasil dihapus!'})
        except User.DoesNotExist:
            return JsonResponse({'success': False, 'message': 'User tidak ditemukan!'})

# ================= CRUD KAPASITAS ADMIN =================
def tambah_area(request):
    if request.session.get('level') != 'admin':
        return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

    if request.method == 'POST':
        kd_area   = request.POST.get('kd_area', '').strip().upper()
        nama_area = request.POST.get('nama_area', '').strip()
        kapasitas = request.POST.get('kapasitas', '').strip()

        if not kd_area or not nama_area or not kapasitas:
            return JsonResponse({'success': False, 'message': 'Semua kolom harus diisi!'})

        if AreaParkir.objects.filter(kd_area=kd_area).exists():
```

```
        return JsonResponse({'success': False, 'message': 'Kode area sudah ada!'})

    AreaParkir.objects.create(kd_area=kd_area, nama_area=nama_area, kapasitas=int(kapasitas))
    return JsonResponse({'success': True, 'message': 'Lahan berhasil ditambahkan!'})


def edit_area(request, kd_area):
  if request.session.get('level') != 'admin':
    return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

  if request.method == 'POST':
    kd_area_baru = request.POST.get('kd_area', '').strip().upper()
    nama_area    = request.POST.get('nama_area', '').strip()
    kapasitas    = request.POST.get('kapasitas', '').strip()

    if not kd_area_baru or not nama_area or not kapasitas:
      return JsonResponse({'success': False, 'message': 'Semua kolom harus diisi!'})

    try:
      area = AreaParkir.objects.get(kd_area=kd_area)

      if kd_area != kd_area_baru:
        if AreaParkir.objects.filter(kd_area=kd_area_baru).exists():
          return JsonResponse({'success': False, 'message': 'Kode area sudah dipakai!'})
        area.delete()
        AreaParkir.objects.create(
          kd_area=kd_area_baru,
          nama_area=nama_area,
          kapasitas=int(kapasitas)
        )
      else:
        area.nama_area = nama_area
        area.kapasitas = int(kapasitas)
        area.save()

      return JsonResponse({'success': True, 'message': 'Lahan berhasil diupdate!'})
    except AreaParkir.DoesNotExist:
      return JsonResponse({'success': False, 'message': 'Lahan tidak ditemukan!'})


def hapus_area(request, kd_area):
  if request.session.get('level') != 'admin':
    return JsonResponse({'success': False, 'message': 'Unauthorized'}, status=403)

  if request.method == 'POST':
    try:
      area = AreaParkir.objects.get(kd_area=kd_area)
      area.delete()
      return JsonResponse({'success': True, 'message': 'Lahan berhasil dihapus!'})
    except AreaParkir.DoesNotExist:
      return JsonResponse({'success': False, 'message': 'Lahan tidak ditemukan!'})

# ================= DASHBOARD OWNER =================
def owner_dashboard(request):
  if request.session.get('level') != 'owner':
    return redirect('login')

  # default: dari awal bulan sampai hari ini
  hari_ini   = timezone.now().date()
  awal_bulan = hari_ini.replace(day=1)

  dari_str   = request.GET.get('dari',   awal_bulan.strftime('%Y-%m-%d'))
  sampai_str = request.GET.get('sampai', hari_ini.strftime('%Y-%m-%d'))
  status_filter = request.GET.get('status', 'SEMUA')

  try:
    dari   = datetime.strptime(dari_str,   '%Y-%m-%d').date()
    sampai = datetime.strptime(sampai_str, '%Y-%m-%d').date()
  except ValueError:
```

```
        dari   = awal_bulan
        sampai = hari_ini

    # filter transaksi
    qs = TransaksiParkir.objects.filter(
        waktu_masuk__date__gte=dari,
        waktu_masuk__date__lte=sampai,
    )
    if status_filter != 'SEMUA':
        qs = qs.filter(status=status_filter)
    transaksi = qs.order_by('-waktu_masuk')

    for t in transaksi:
        t.durasi_display, t.biaya_display = hitung_durasi_biaya(t.waktu_masuk, t.waktu_keluar)

    # log aktivitas terbaru
    log_aktivitas = TransaksiParkir.objects.filter(status='DONE').order_by('-waktu_keluar')[:50]
    for l in log_aktivitas:
        l.durasi_display, l.biaya_display = hitung_durasi_biaya(l.waktu_masuk, l.waktu_keluar)

    # grafik
    tahun_ini = timezone.now().year
    pendapatan_bulanan = (
        TransaksiParkir.objects
        .filter(status='DONE', waktu_keluar__year=tahun_ini)
        .annotate(bulan=TruncMonth('waktu_keluar'))
        .values('bulan')
        .annotate(total=Sum('biaya'))
        .order_by('bulan')
    )
    bulan_labels = ['Jan','Feb','Mar','Apr','Mei','Jun','Jul','Agu','Sep','Okt','Nov','Des']
    grafik_labels, grafik_data = [], []
    for p in pendapatan_bulanan:
        grafik_labels.append(bulan_labels[p['bulan'].month - 1])
        grafik_data.append(p['total'] or 0)

    jumlah_in   = TransaksiParkir.objects.filter(status='IN').count()
    jumlah_done = TransaksiParkir.objects.filter(status='DONE').count()

    context = {
        'transaksi':     transaksi,
        'dari':          dari_str,
        'sampai':        sampai_str,
        'status_filter': status_filter,
        'log_aktivitas': log_aktivitas,
        'grafik_labels': grafik_labels,
        'grafik_data':   grafik_data,
        'jumlah_in':     jumlah_in,
        'jumlah_done':   jumlah_done,
    }
    return render(request, 'owner/owner_dashboard.html', context)

# ================= EXPORT PDF OWNER =================
def export_pdf_owner(request):
    if request.session.get('level') != 'owner':
        return redirect('login')

    hari_ini   = timezone.now().date()
    awal_bulan = hari_ini.replace(day=1)

    dari_str     = request.GET.get('dari',   awal_bulan.strftime('%Y-%m-%d'))
    sampai_str   = request.GET.get('sampai', hari_ini.strftime('%Y-%m-%d'))
    status_filter = request.GET.get('status', 'SEMUA')

    try:
        dari   = datetime.strptime(dari_str,   '%Y-%m-%d').date()
        sampai = datetime.strptime(sampai_str, '%Y-%m-%d').date()
    except ValueError:
        dari   = awal_bulan
```

```
        sampai = hari_ini

    qs = TransaksiParkir.objects.filter(
        waktu_masuk__date__gte=dari,
        waktu_masuk__date__lte=sampai,
    )
    if status_filter != 'SEMUA':
        qs = qs.filter(status=status_filter)
    transaksi = qs.order_by('-waktu_masuk')

    response = HttpResponse(content_type='application/pdf')
    response['Content-Disposition'] = f'attachment; filename="transaksi_{dari_str}_sd_{sampai_str}.pdf"'

    doc   = SimpleDocTemplate(response, pagesize=A4)
    style = getSampleStyleSheet()
    elems = []

    elems.append(Paragraph("Laporan Transaksi Parkir", style['Title']))
    elems.append(Paragraph(f"Periode: {dari_str} s/d {sampai_str} | Status: {status_filter}", style['Normal']))
    elems.append(Spacer(1, 12))

    data = [['No', 'Plat No', 'Area', 'Jam Masuk', 'Jam Keluar', 'Durasi', 'Biaya', 'Status']]
    for i, t in enumerate(transaksi, 1):
        durasi, biaya = hitung_durasi_biaya(t.waktu_masuk, t.waktu_keluar)
        data.append([
            str(i), t.noPlat, t.kd_area.kd_area,
            t.waktu_masuk.strftime('%d/%m %H:%M'),
            t.waktu_keluar.strftime('%d/%m %H:%M') if t.waktu_keluar else '-',
            f"{durasi} jam", f"Rp {biaya:,}", t.status
        ])

    tabel = Table(data, repeatRows=1)
    tabel.setStyle(TableStyle([
        ('BACKGROUND',     (0,0), (-1,0), colors.HexColor('#111111')),
        ('TEXTCOLOR',      (0,0), (-1,0), colors.white),
        ('FONTSIZE',       (0,0), (-1,0), 9),
        ('FONTSIZE',       (0,1), (-1,-1), 8),
        ('ROWBACKGROUNDS', (0,1), (-1,-1), [colors.white, colors.HexColor('#fff8e1')]),
        ('GRID',           (0,0), (-1,-1), 0.5, colors.HexColor('#e0e0e0')),
        ('ALIGN',          (0,0), (-1,-1), 'CENTER'),
        ('VALIGN',         (0,0), (-1,-1), 'MIDDLE'),
        ('PADDING',        (0,0), (-1,-1), 6),
    ]))
    elems.append(tabel)
    doc.build(elems)
    return response

# ================= LOGOUT =================
def logout_view(request):
    username = request.session.get('username')
    if username:
        broadcast_petugas_logout(username)
    request.session.flush()
    return redirect('login')
```

B. Mqtt_client.py

```
import paho.mqtt.client as mqtt
import json
import math
import threading
from django.utils import timezone
from .models import TransaksiParkir, AreaParkir
from .models import StrukQueue

BROKER       = "localhost"
PORT         = 1883
NAMA_PETUGAS = "dhea"

TOPIC_CONFIG       = f"parking/{NAMA_PETUGAS}/config/petugas"
```

```
TOPIC_ENTRY_RFID   = f"parking/{NAMA_PETUGAS}/entry/rfid"
TOPIC_EXIT_RFID    = f"parking/{NAMA_PETUGAS}/exit/rfid"
TOPIC_ENTRY_SERVO  = f"parking/{NAMA_PETUGAS}/entry/servo"
TOPIC_EXIT_SERVO   = f"parking/{NAMA_PETUGAS}/exit/servo"
TOPIC_LCD          = f"parking/{NAMA_PETUGAS}/lcd"

# Urutan lahan parkir
URUTAN_LAHAN = ["BSP", "LKW", "TXS"]

_mqtt_client   = None
_petugas_aktif = None
_processing    = False

def set_petugas_aktif(username):
    global _petugas_aktif
    _petugas_aktif = username

def get_petugas_aktif():
    return _petugas_aktif

def _hitung_biaya(waktu_masuk, waktu_keluar=None):
    waktu_akhir = waktu_keluar if waktu_keluar else timezone.now()
    durasi_jam  = math.ceil((waktu_akhir - waktu_masuk).total_seconds() / 3600) or 1
    return durasi_jam, durasi_jam * 2000

def _format_nominal(biaya):
    return f"{biaya:,}".replace(",", ".")

def _lcd(line1, line2=""):
    _publish(TOPIC_LCD, f"{line1}|{line2}")

# ===================== ALOKASI LAHAN =====================
def _get_area_tersedia():
    for kd in URUTAN_LAHAN:
        try:
            area   = AreaParkir.objects.get(kd_area=kd)
            terisi = TransaksiParkir.objects.filter(kd_area=area, status__in=["IN", "OUT"]).count()
            if terisi < area.kapasitas:
                return area
        except AreaParkir.DoesNotExist:
            continue
    return None

# ===================== CALLBACK =====================
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[MQTT] Connected to Mosquitto Broker")
        client.subscribe(TOPIC_ENTRY_RFID)
        client.subscribe(TOPIC_EXIT_RFID)
        print(f"[MQTT] Subscribe: {TOPIC_ENTRY_RFID}")
        print(f"[MQTT] Subscribe: {TOPIC_EXIT_RFID}")
    else:
        print(f"[MQTT] Gagal connect, rc={rc}")

def on_message(client, userdata, msg):
    global _processing

    if _processing:
        print("[MQTT] ⚠ Pesan diabaikan, masih proses...")
        return

    _processing = True

    topic = msg.topic
    try:
        payload = json.loads(msg.payload.decode())
    except json.JSONDecodeError:
        payload = {"rfid": msg.payload.decode().strip()}
```

```
    print(f"[MQTT] {topic} → {payload}")

    card_id = payload.get("rfid", "").upper()
    if not card_id:
        _processing = False
        return

    if topic == TOPIC_ENTRY_RFID:
        handle_masuk(card_id)
    elif topic == TOPIC_EXIT_RFID:
        handle_keluar(card_id)

    _processing = False

# ===================== BROADCAST =====================
def broadcast_petugas_login(username):
    set_petugas_aktif(username)
    _publish(TOPIC_CONFIG, json.dumps({"petugas": username}))
    print(f"[MQTT] Broadcast login: {username}")

def broadcast_petugas_logout(username):
    set_petugas_aktif(None)
    _publish(TOPIC_CONFIG, json.dumps({"petugas": "unknown"}))
    print(f"[MQTT] Broadcast logout: {username}")

# ===================== HANDLER MASUK =====================
def handle_masuk(card_id):
    sedang_parkir = TransaksiParkir.objects.filter(
        card_id=card_id,
        status__in=["IN", "OUT"]
    ).exists()

    if sedang_parkir:
        _lcd("Kendaraan", "Sedang Parkir")
        print(f"[MASUK] Ditolak - {card_id} sudah parkir")
        print(f"═══ LOG: Kartu {card_id} ditolak masuk (sudah parkir) ═══")
        return

    area = _get_area_tersedia()
    if not area:
        _lcd("Lahan Parkir", "Sudah Penuh!")
        print(f"[MASUK] Semua lahan penuh")
        return

    transaksi = TransaksiParkir.objects.create(
        card_id        = card_id,
        kd_area        = area,
        waktu_masuk    = timezone.now(),
        jenis_kendaraan = "Motor",
        status         = "IN",
    )

    _publish(TOPIC_ENTRY_SERVO, "OPEN")
    _lcd("Selamat Datang", transaksi.kd_transaksi)

    print(f"═══════════════════════════════════════════════════════")
    print(f"  ✅ KENDARAAN MASUK")
    print(f"  Card ID   : {card_id}")
    print(f"  Area      : {area.nama_area}")
    print(f"  Kode      : {transaksi.kd_transaksi}")
    print(f"  Waktu     : {transaksi.waktu_masuk.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"═══════════════════════════════════════════════════════")

# ===================== HANDLER KELUAR =====================
def handle_keluar(card_id):
    transaksi = TransaksiParkir.objects.filter(
        card_id=card_id,
        status="IN"
    ).first()
```

```
    if not transaksi:
        _lcd("Kendaraan", "Tidak Parkir")
        print(f"[KELUAR] Ditolak - {card_id} tidak sedang parkir")
        print(f"═══ LOG: Kartu {card_id} ditolak keluar (tidak sedang parkir) ═══")
        return

    waktu_keluar      = timezone.now()
    durasi_jam, biaya = _hitung_biaya(transaksi.waktu_masuk, waktu_keluar)
    nominal           = _format_nominal(biaya)

    transaksi.waktu_keluar = waktu_keluar
    transaksi.biaya        = biaya
    transaksi.status       = "OUT"
    transaksi.save()

    nama_area = transaksi.kd_area.nama_area
    kd        = transaksi.kd_transaksi

    struk = f"""
STRUK AUTOPARKING
--------------------------------------------
Kode Transaksi  : {kd}
ID Transaksi    : {transaksi.id_transaksi}
ID Card         : {card_id}
Area Parkir     : {nama_area}
Masuk           : {transaksi.waktu_masuk.strftime('%Y-%m-%d %H:%M:%S')}
Keluar          : {waktu_keluar.strftime('%Y-%m-%d %H:%M:%S')}
Lama Parkir     : {durasi_jam} jam
Biaya Parkir    : Rp {nominal}
--------------------------------------------
"""
    StrukQueue.objects.create(kd_transaksi=kd, struk=struk)

    _lcd(f"Rp {nominal}", f"{durasi_jam} jam")

    print(f"═══════════════════════════════════════════════════════════════")
    print(f"  🚗 KENDARAAN KELUAR (menunggu approve)")
    print(f"  Card ID   : {card_id}")
    print(f"  Area      : {nama_area}")
    print(f"  Kode      : {kd}")
    print(f"  Masuk     : {transaksi.waktu_masuk.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"  Keluar    : {waktu_keluar.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"  Durasi    : {durasi_jam} jam")
    print(f"  Biaya     : Rp {nominal}")
    print(f"  ⏳ Menunggu petugas approve di web...")
    print(f"═══════════════════════════════════════════════════════════════")

# ===================== CETAK STRUK =====================
def pop_struk_terbaru():
    from .models import StrukQueue
    struk = StrukQueue.objects.filter(sudah_diambil=False).order_by('created_at').first()
    if struk:
        struk.sudah_diambil = True
        struk.save()
        return {"kd_transaksi": struk.kd_transaksi, "struk": struk.struk}
    return None

# ===================== APPROVE KELUAR =====================
def approve_keluar(kd_transaksi):
    try:
        transaksi = TransaksiParkir.objects.get(kd_transaksi=kd_transaksi, status="OUT")
    except TransaksiParkir.DoesNotExist:
        return False, "Transaksi tidak ditemukan"

    transaksi.status = "DONE"
    transaksi.save()

    durasi_jam, biaya = _hitung_biaya(transaksi.waktu_masuk, transaksi.waktu_keluar)
```

```
    nominal          = _format_nominal(biaya)
    nama_area        = transaksi.kd_area.nama_area
    kd               = transaksi.kd_transaksi

    _publish(TOPIC_EXIT_SERVO, "OPEN")
    _lcd("Selamat Jalan!", f"Rp {nominal}")

    print(f"═══════════════════════════════════════════════════════════")
    print(f"  ✅ APPROVE KELUAR - PALANG DIBUKA")
    print(f"  Kode     : {kd}")
    print(f"  Area     : {nama_area}")
    print(f"  Durasi   : {durasi_jam} jam")
    print(f"  Biaya    : Rp {nominal}")
    print(f"═══════════════════════════════════════════════════════════")

    return True, "Berhasil"

# ===================== REJECT KELUAR =====================
def reject_keluar(kd_transaksi):
    try:
        transaksi = TransaksiParkir.objects.get(kd_transaksi=kd_transaksi, status="OUT")
    except TransaksiParkir.DoesNotExist:
        return False, "Transaksi tidak ditemukan"

    transaksi.status      = "IN"
    transaksi.waktu_keluar = None
    transaksi.biaya       = 0
    transaksi.save()

    _lcd("Akses Ditolak", "Hub. Petugas")

    print(f"═══════════════════════════════════════════════════════════")
    print(f"  ❌ REJECT KELUAR")
    print(f"  Kode     : {kd_transaksi}")
    print(f"  Status   : Dikembalikan ke IN")
    print(f"═══════════════════════════════════════════════════════════")

    return True, "Ditolak"

# ===================== PUBLISH HELPER =====================
def _publish(topic, payload):
    global _mqtt_client
    if _mqtt_client and _mqtt_client.is_connected():
        _mqtt_client.publish(topic, payload)
    else:
        import random, string
        unique_id = ''.join(random.choices(string.ascii_lowercase + string.digits, k=6))
        c = mqtt.Client(client_id=f"pub-{unique_id}")
        c.connect(BROKER, PORT, 60)
        c.publish(topic, payload)
        c.disconnect()

def publish_message(message):
    _publish("parking/gate", message)

# ===================== START MQTT =====================
def start_mqtt():
    global _mqtt_client

    if _mqtt_client and _mqtt_client.is_connected():
        print("[MQTT] Sudah konek, skip!")
        return _mqtt_client

    import random, string
    unique_id = ''.join(random.choices(string.ascii_lowercase + string.digits, k=6))
    client = mqtt.Client(client_id=f"django-parkir-{unique_id}", clean_session=True)
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, PORT, keepalive=60)
```

```
    _mqtt_client = client
    threading.Thread(target=client.loop_forever, daemon=True).start()
    print("[MQTT] Client started (Mosquitto localhost:1883)")
    return client
```

C. Sketch.ino

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ================= WIFI =================
const char* ssid     = "2001!";
const char* password = "wonwon17";

// ================= MQTT =================
const char* mqtt_server  = "10.127.156.195";
const int   mqtt_port    = 1883;
const char* nama_petugas = "dhea";

// topic
String entryTopic      = "parking/" + String(nama_petugas) + "/entry/rfid";
String exitTopic       = "parking/" + String(nama_petugas) + "/exit/rfid";
String servoEntryTopic = "parking/" + String(nama_petugas) + "/entry/servo";
String servoExitTopic  = "parking/" + String(nama_petugas) + "/exit/servo";
String lcdTopic        = "parking/" + String(nama_petugas) + "/lcd";

WiFiClient   espClient;
PubSubClient client(espClient);

// ================= OLED =================
#define SCREEN_WIDTH  128
#define SCREEN_HEIGHT 64
#define OLED_ADDR     0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ================= RFID =================
#define SS_MASUK   5
#define RST_MASUK  27
#define SS_KELUAR  15
#define RST_KELUAR 17

MFRC522 rfidMasuk(SS_MASUK, RST_MASUK);
MFRC522 rfidKeluar(SS_KELUAR, RST_KELUAR);

// ================= SERVO =================
#define SERVO_MASUK  32
#define SERVO_KELUAR 33

Servo servoMasuk;
Servo servoKeluar;

// ================= OLED HELPER =================
void showOLED(String baris1, String baris2 = "") {
  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextSize(2);
  display.setCursor(0, 10);
  display.println(baris1);
  if (baris2 != "") {
    display.setTextSize(1);
    display.setCursor(0, 45);
    display.println(baris2);
  }
```

```
  display.display();
}

// ================= BACA UID =================
String readRFID(MFRC522 &rfid) {
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();
  return uid;
}

// ================= MQTT CALLBACK =================
void callback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)payload[i];

  String t = String(topic);
  Serial.println("[MQTT] " + t + " → " + msg);

  // --- OLED dari Django ---
  if (t == lcdTopic) {
    // Format dari Python: "baris1|baris2"
    int sep = msg.indexOf('|');
    if (sep != -1) {
      showOLED(msg.substring(0, sep), msg.substring(sep + 1));
    } else {
      showOLED(msg);
    }
    delay(3000);
    showOLED("Silahkan", "Tempelkan Kartu");
  }

  // --- Servo Masuk (tap masuk langsung buka) ---
  if (t == servoEntryTopic && msg == "OPEN") {
    Serial.println("[SERVO] Palang Masuk BUKA");
    servoMasuk.write(0);
    delay(2000);
    servoMasuk.write(90);
    Serial.println("[SERVO] Palang Masuk TUTUP");
  }

  // --- Servo Keluar (hanya buka kalau petugas approve) ---
  if (t == servoExitTopic && msg == "OPEN") {
    Serial.println("[SERVO] Palang Keluar BUKA");
    servoKeluar.write(0);
    delay(2000);
    servoKeluar.write(90);
    Serial.println("[SERVO] Palang Keluar TUTUP");
  }
}

// ================= MQTT RECONNECT =================
void reconnect() {
  while (!client.connected()) {
    Serial.println("[MQTT] Connecting...");
    String clientId = "ESP32-" + String(random(9999));
    if (client.connect(clientId.c_str())) {
      Serial.println("[MQTT] Connected!");
      client.subscribe(servoEntryTopic.c_str());
      client.subscribe(servoExitTopic.c_str());
      client.subscribe(lcdTopic.c_str());
      Serial.println("[MQTT] Subscribe OK");
    } else {
      Serial.println("[MQTT] Gagal, coba lagi...");
      delay(2000);
    }
```

```
  }
}

// ================= SETUP =================
void setup() {
  Serial.begin(115200);

  // OLED
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  showOLED("Silahkan", "Tempelkan Kartu");
  Serial.println("[OLED] OK");

  // RFID
  SPI.begin(18, 19, 23);
  rfidMasuk.PCD_Init();
  rfidKeluar.PCD_Init();
  Serial.println("[RFID] OK");

  // Servo
  servoMasuk.attach(SERVO_MASUK);
  servoKeluar.attach(SERVO_KELUAR);
  servoMasuk.write(90);
  servoKeluar.write(90);
  Serial.println("[SERVO] OK");

  // WiFi
  showOLED("Connecting", "WiFi...");
  WiFi.begin(ssid, password);
  Serial.print("[WiFi] Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[WiFi] Connected! IP: " + WiFi.localIP().toString());
  showOLED("WiFi OK!", WiFi.localIP().toString());
  delay(2000);
  showOLED("Silahkan", "Tempelkan Kartu");

  // MQTT
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
}

/// ================= LOOP =================
bool sudahKirimMasuk = false;
bool sudahKirimKeluar = false;

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  // --- RFID MASUK ---
  if (rfidMasuk.PICC_IsNewCardPresent() && rfidMasuk.PICC_ReadCardSerial()) {
    if (!sudahKirimMasuk) {
      sudahKirimMasuk = true;
      String uid = readRFID(rfidMasuk);
      Serial.println("[MASUK] UID: " + uid);

      String payload = "{\"rfid\":\"" + uid + "\"}";
      client.publish(entryTopic.c_str(), payload.c_str());
      showOLED("Memproses...", uid);

      rfidMasuk.PICC_HaltA();
      rfidMasuk.PCD_StopCrypto1();
      delay(3000);
      sudahKirimMasuk = false;
    }
  }
```

```
// --- RFID KELUAR ---
if (rfidKeluar.PICC_IsNewCardPresent() && rfidKeluar.PICC_ReadCardSerial()) {
  if (!sudahKirimKeluar) {
    sudahKirimKeluar = true;
    String uid = readRFID(rfidKeluar);
    Serial.println("[KELUAR] UID: " + uid);

    String payload = "{\"rfid\":\"" + uid + "\"}";
    client.publish(exitTopic.c_str(), payload.c_str());
    showOLED("Memproses...", uid);

    rfidKeluar.PICC_HaltA();
    rfidKeluar.PCD_StopCrypto1();
    delay(3000);
    sudahKirimKeluar = false;
  }
 }
}
```

10. PENGUJIAN

A. Login

No	Skenario Penguji	Test Case	Hasil yang diharapkan	Hasil Pengujian	Kesimpulan
1	Username dan password tidak diisi	Username (kosong) password (kosong)	System akan menolak dan menampilkan pesan "Please fill out this field."	Sesuai harapan	Valid
2	Mengisi username, dan password tidak diisi	Username (erik1) password (kosong)	System akan menolak dan menampilkan pesan "Please fill out this field."	Sesuai harapan	Valid
3	Mengisi password, dan username tidak diisi	username (kosong) password (erik123)	System akan menolak dan menampilkan pesan "Please fill out this field."	Sesuai harapan	Valid
4	Mengisi username dan password tidak sesuai	username (erik1) password (fred33)	System akan menolak dan menampilkan pesan "Username dan Password salah!"	Sesuai harapan	Valid
5	Mengisi username dan password yang sesuai	username (erik1) password (erik123)	System menerima akses login dan menampilkan halaman utama petugas	Sesuai harapan	Valid

B. CRUD User (tambah)

No	Skenario Penguji	Test Case	Hasil yang diharapkan	Hasil Pengujian	Kesimpulan
1	Username, role, dan password tidak diisi.	Username (kosong) Role (kosong) Password (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
2	Mengisi Username, tetapi role dan password tidak diisi.	Username (greg) Role (kosong) Password (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
3	Mengisi Username, mengisi role tetapi password tidak diisi.	Username (greg) Role (owner) Password (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
4	Tidak mengisi Username, tetapi role dan password diisi,	Username (kosong) Role (owner) Password (greg123)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
5	Mengisi Username, role, dan password	Username (greg) Role (owner) Password (greg123)	System menerima data dan menampilkan data tersebut dalam tabel	Sesuai harapan	Valid

C. CRUD Lahan Parkir (tambah)

No	Skenario Penguji	Test Case	Hasil yang diharapkan	Hasil Pengujian	Kesimpulan
1	kode area, nama area, dan kapasitas tidak diisi.	Kode area (kosong) Nama area (kosong) Kapasitas (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid

2	Mengisi kode area, tetapi nama area dan kapasitas tidak diisi.	Kode area (VTP) Nama area (kosong) Kapasitas (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
3	Mengisi kode area, dan nama area, tetapi kapasitas tidak diisi.	Kode area (VTP) Nama area (RGT) Kapasitas (kosong)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
4	Kode area dan kapasitas diisi, tetapi nama area tidak	Kode area (VTP) Nama area (kosong) Kapasitas (35)	System akan menolak dan menampilkan pesan "Semua kolom harus diisi!"	Sesuai harapan	Valid
5	Mengisi kode area, nama area, dan kapasitas	Kode area (VTP) Nama area (RGT) Kapasitas (35)	System menerima data dan menampilkan data tersebut dalam tabel	Sesuai harapan	Valid

11. UPLOAD GITHUB

https://github.com/ZaxkChEw/AutoParking-UKK.git

12. PEMBUATAN MANUAL BOOK

13. REVISI